

Чтение с середины индекса

Когда возникает

Вроде и прочитали немного, и все по индексу, и никого лишнего не фильтровали — а все равно прочитано существенно больше страниц, чем хотелось бы.

Как опознать

```
1 -> Index [Only] Scan [Backward]
2   && loops × (rows + RRbF) < (shared hit + shared read) × 8
3   -- прочитано больше 1KB на каждую запись
4   && shared hit + shared read > 64
```

Рекомендации

Внимательно посмотреть на структуру использовавшегося индекса и ключевые поля, заданные в запросе — скорее всего, часть индекса не задана. Скорее всего, вам придется создать похожий индекс, но без префиксных полей или [научиться итерировать их значения](#).

Пример:

```
1 CREATE TABLE tbl AS
2 SELECT
3   generate_series(1, 100000) pk      -- 100K "фактов"
4   , (random() * 100)::integer fk_org  -- 100 разных внешних ключей
5   , (random() * 1000)::integer fk_cli; -- 1K разных внешних ключей
6
7 CREATE INDEX ON tbl(fk_org, fk_cli); -- все почти как в #2
8 -- только вот отдельный индекс по fk_cli мы уже посчитали лишним и
  удалили
9
10 SELECT
11   *
12 FROM
13   tbl
14 WHERE
15   fk_cli = 999 -- а fk_org не задано, хотя стоит в индексе раньше
16 LIMIT 20;
```

#	node	ms	tree	ms	rows	ratio	node	sh.ht
		0.486		20			итоговые результаты	83
0	0.005	0.486		20			Limit	
1	0.481	0.481		20			-> Index Scan using tbl_fk_org_fk_cli_idx on tbl	83

Вроде бы все хорошо, даже по индексу, но как-то подозрительно — на каждую из 20 прочитанных записей пришлось вычитать по 4 страницы данных, 32KB на запись — не жирно ли? Да и имя индекса `tbl_fk_org_fk_cli_idx` наводит на размышления.

Исправляем:

```
CREATE INDEX ON tbl(fk_cli);
```

#	node, ms	tree, ms	rows	ratio	node	sh,ht
		0.046	20		итоговые результаты	22
0	0.003	0.046	20		Limit	
1	0.043	0.043	20		-> Index Scan using tbl_fk_cli_idx on tbl	22

Внезапно — в 10 раз быстрее, и в 4 раза меньше читать!

Другие примеры ситуаций неэффективного использования индексов можно увидеть в статье [DBA: находим бесполезные индексы](#).